



Appunti di Architettura degli Elaboratori

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	Teoria del corso	5
1	Sistemi Numerici	6
1.1	Introduzione	6
1.2	Bit	6
1.2.1	Definizione	6
1.2.2	Unità di misura	6
1.2.3	Numero di configurazione possibili	6
1.2.4	Numero massimo rappresentabile	6
1.3	Rappresentazione di un sistema numerico posizionale	7
1.3.1	Cosa è un sistema numerico posizionale?	7
1.3.2	Formula	7
1.4	Sistema Binario	7
1.4.1	Definizione	7
1.4.2	Esempio	7
1.4.3	Byte	7
1.5	Sistema Ottale	8
1.5.1	Definizione	8
1.5.2	Esempio	8
1.6	Sistema Esadecimale	8
1.6.1	Definizione	8
1.6.2	Significato delle lettere	8
1.6.3	Compattezza	8
1.7	Conversione tra sistemi numerici	9
1.7.1	Da base $r \rightarrow$ a base 10	9
1.7.2	Da base 10 $\rightarrow$ a base r	9
1.7.3	Da base $r \rightarrow$ a base r'	10
1.8	Rappresentazioni dei valori	12
1.8.1	Limite di cifre	12
1.8.2	Tabella numeri rappresentabili	12
1.8.3	Prefissi	12
2	Sistema binario: Operazioni aritmetiche	13
2.1	Somma	13
2.1.1	Definizione	13
2.1.2	Esempio	13
2.1.3	Esempio con Overflow	14
2.2	Sottrazione	15
2.2.1	Definizione	15
2.2.2	Esempio	15
2.2.3	Esempio di Sottrazione Impossibile (Underflow)	15
2.3	Shift	16
2.3.1	Definizione	16
2.3.2	Left Shift	16
2.3.3	Right Shift	16
3	Rappresentazione dei numeri negativi	17

3.1	Perchè ne abbiamo bisogno?	17
3.2	MS - Modulo e Segno	17
3.2.1	Procedura	17
3.2.2	Range di rappresentabilità	17
3.2.3	Difetti del sistema	17
3.2.4	Esempio di conversione	17
3.2.5	Somma	18
3.2.6	Sottrazione	18
3.3	CA1 - Complemento a 1	20
3.3.1	Complemento	20
3.3.2	Procedura	20
3.3.3	Range di numeri rappresentabili	20
3.3.4	Difetti	20
3.3.5	Esempi di conversione	20
3.4	CA2 - Complemento a 2	21
3.4.1	Procedura	21
3.4.2	Conversione a decimale	21
3.4.3	Range di numeri rappresentabili	21
3.4.4	Esempi di conversione	21
3.4.5	Somma	21
3.4.6	Sottrazione	22
3.5	Rappresentazione eccesso 2^{n-1}	23
3.5.1	Procedure	23
3.5.2	Range numeri rappresentabili	23
3.5.3	Esempio di conversione	23
3.6	Rappresentazione Eccesso 128	24
3.6.1	Procedura	24
3.6.2	Esempi	24
3.6.3	Osservazioni	24
4	Rappresentazione numeri frazionari con la virgola fissa	25
4.1	Idea	25
4.2	UNSIGNED fixed point	25
4.2.1	Cosa è?	25
4.2.2	Struttura del formato	25
4.2.3	Range di valori rappresentabili	25
4.2.4	Peso delle posizioni	26
4.2.5	Da Decimale a Virgola Fissa Unsigned	26
4.2.6	Da Virgola Fissa Unsigned a Decimale	27
4.3	SIGNED fixed point	28
4.3.1	Struttura del formato	28
4.3.2	Peso delle posizioni	28
4.3.3	Da numero decimale positivo a Signed Fixed-Point	28
4.3.4	Da numero decimale negativo a Signed Fixed-Point	29
5	Rappresentazione numeri frazionari con la virgola mobile	30
5.1	Cosa è?	30
5.2	Struttura	30
5.2.1	32 bit	30
5.2.2	64 bit	30
5.3	Formule	30
5.3.1	Formula della rappresentazione in virgola mobile	30
5.3.2	Normalizzazione e "1 implicito"	31

5.4	Conversione da Decimale a Virgola Mobile	31
5.5	Conversione da Virgola Mobile a Decimale	32
5.6	Errore assoluto E_a	32
5.6.1	Cosa è?	32
5.6.2	Formula	32
5.6.3	Esempio di calcolo	32
5.7	Errore Relativo E_r	33
5.7.1	Cos'è?	33
5.7.2	Formula	33
5.7.3	Esempio di calcolo	33
6	Rappresentazione caratteri	34
6.1	Idea	34
6.2	ASCII Standard	34
6.2.1	Caratteristiche	34
6.2.2	Tabella	35
6.3	ASCII Esteso	36
6.3.1	Caratteristiche	36
6.3.2	Tabella	36
6.4	UNICODE	37
6.4.1	Caratteristiche	37
7	Instruction Set Architecture (ISA)	38
7.1	Cosa è?	38
7.2	Architettura MIPS	38
7.2.1	Caratteristiche	38
7.3	Formati delle istruzioni	38
7.4	Rappresentazioni disponibili	38
7.4.1	Stringa a 32 bit	38
7.4.2	Stringa a 32 bit sezionata	38
7.4.3	Rappresentazione esadecimale	38
7.4.4	Formato assembly	39
8	ISA R-Type	40
8.1	Cosa sono?	40
8.2	Struttura	40
8.2.1	Struttura della stringa a 32 bit	40
8.2.2	Spiegazione dei campi	40
9	ISA I-Type	41
9.1	Cosa sono?	41
9.2	Struttura	41
9.2.1	Struttura della stringa a 32 bit	41
9.2.2	Spiegazione dei campi	41
9.3	Istruzione load word	42
9.3.1	Struttura stringa a 32 bit	42
9.3.2	Spiegazione dei campi	42
9.3.3	Svolgimento dell'operazione	42
9.4	Istruzione store word	43
9.4.1	Struttura stringa a 32 bit	43
9.4.2	Spiegazione dei campi	43
9.4.3	Svolgimento dell'operazione	43
9.5	Istruzioni di branch	44
9.5.1	Struttura stringa a 32 bit	44

9.5.2	Spiegazione dei campi	44
9.5.3	Spiegazione dell'operazione	44
10	ISA J-Type	45
10.1	Cosa sono?	45
10.2	Struttura	45
10.2.1	Struttura della stringa a 32 bit	45
10.2.2	Spiegazione dei campi	45
11	Assembly	46
11.1	Linguaggi di alto e basso livello	46
11.1.1	Linguaggio assembler	46
11.1.2	Linguaggi ad alto livello	46
11.2	Catena programmatica (Vista ad alto livello)	47
11.2.1	Struttura	47
11.2.2	Ruolo del Compiler	47
11.3	Catena programmatica (Vista a basso livello)	47
11.3.1	Struttura	47
11.3.2	Assembler	48
11.3.3	Linker (Link editor)	49
11.4	Convenzioni nomi ed uso dei registri	50
11.5	Direttive all'assemblatore	50
11.5.1	Cosa sono?	50
11.5.2	Esempi di direttive	50
11.5.3	Esempio d'uso	51
11.6	Pseudoistruzioni	51
11.6.1	Cosa sono?	51
11.6.2	Esempio 1	51
11.6.3	Esempio 2	51
11.7	Flusso di controllo	52
11.7.1	Problema	52
11.7.2	Istruzione jal - Jump And Link	52
11.7.3	Istruzione jr - Jump Register	53
11.8	Syscall	53
11.8.1	Cosa sono?	53
11.8.2	Come usare le syscall	54
11.8.3	Tabella	54
11.8.4	Esempio d'uso senza procedure	55
11.8.5	Esempio d'uso con procedure	56
2	Esercitazioni svolte	57
3	Laboratori svolti	58
4	Quiz Mega svolti	59
5	Domande aperte Mega svolte	60

Parte 1

Teoria del corso

1. Sistemi Numerici

1.1 Introduzione

L'aritmetica svolta dai computer differisce dall'aritmetica a noi familiare, i motivi sono molteplici, ma quello principale è sicuramente da attribuire alla diversa modalità nella rappresentazione dei dati e delle informazioni.

$16_{10} \neq 16_{16}$ (16 in decimale è diverso da 16 in esadecimale)

$$16_{10} = 1 \cdot 10^1 + 6 \cdot 10^0 = 16_{10}$$

$$16_{16} = 1 \cdot 16^1 + 6 \cdot 16^0 = 22_{10}$$

Se per noi il sistema decimale può sembrare scontato, non è certo lo stesso per i computer, infatti, i calcolatori lavorano sfruttando il sistema binario.

1.2 Bit

1.2.1 Definizione

Con il termine **bit** definiamo l'unità di misura dell'informazione, un bit può assumere solo il valore di 0 o 1

1.2.2 Unità di misura

Combinando tra loro più bit si ottengono strutture complesse, in particolare:

- Byte - 8 bit
- Nybble - 4 bit
- Word - 32 bit
- Halfword - 16 bit
- Doubleword - 64 bit

1.2.3 Numero di configurazione possibili

Dati **k bit** il numero di configurazione ottenibili è pari a 2^k

1.2.4 Numero massimo rappresentabile

Dati **k bit** il massimo numero rappresentabile è pari a $2^k - 1$

1.3 Rappresentazione di un sistema numerico posizionale

1.3.1 Cosa è un sistema numerico posizionale?

È un sistema numerico dove ogni cifra assume un valore diverso a seconda della posizione che occupa all'interno del numero

1.3.2 Formula

$$N = \sum_{i=-m}^{n-1} d_i \cdot r^i$$

d - rappresenta la singola cifra

r - è la radice o base del sistema

n - è il numero di cifre della parte intera (sinistra della virgola)

m - è il numero di cifre della parte frazionaria (destra della virgola)

Esempio d'uso formula

$$15 = 10 \cdot 10^1 + 5 \cdot 10^0$$

1.4 Sistema Binario

1.4.1 Definizione

- Base: $r = 2$
- Cifre: $d = 0, 1$
- Un valore numerico N si rappresenta come:

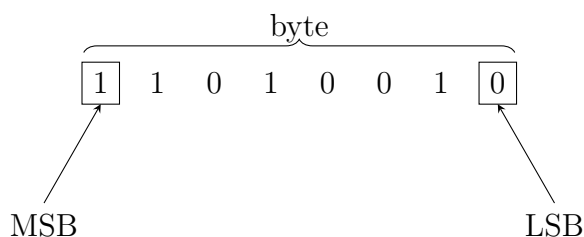
$$N = d_{n-1} \cdot 2^{n-1} + \dots + d_0 \cdot 2^0 + d_{-1} \cdot 2^{-1} + \dots + d_{-m} \cdot 2^{-m}$$

1.4.2 Esempio

$$101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5_{10}$$

1.4.3 Byte

È una sequenza di otto bit consecutivi



- Most Significant Bit (MSB):
il bit più a sinistra
- Least Significant Bit (LSB):
il bit più a destra

1.5 Sistema Ottale

1.5.1 Definizione

- Base $r = 8$
- Cifre $d = 0, 1, \dots, 7$
- Un valore numerico N si rappresenta come:

$$N = d_{n-1} \cdot 8^{n-1} + \dots + d_0 \cdot 8^0 + d_{-1} \cdot 8^{-1} + \dots + d_{-m} \cdot 8^{-m}$$

1.5.2 Esempio

$$127_8 = 1 \cdot 8^2 + 2 \cdot 8^1 + 7 \cdot 8^0 = 87_{10}$$

1.6 Sistema Esadecimale

1.6.1 Definizione

- Base $r = 16$
- Cifre $d = 0, 1, \dots, 9, A, B, C, D, E, F$
- Un valore numerico N si rappresenta come:

$$N = d_{n-1} \cdot 16^{n-1} + \dots + d_0 \cdot 16^0 + d_{-1} \cdot 16^{-1} + \dots + d_{-m} \cdot 16^{-m}$$

Notazione

Spesso si usa il pendice_H al posto del pendice₁₆ per indicare la base decimale oppure $0x$ davanti al numero.

Il sistema esadecimale viene spesso abbreviato come **esa** oppure **hex**

1.6.2 Significato delle lettere

$$A_H = 10_{10} = 1010_2$$

$$B_H = 11_{10} = 1011_2$$

$$C_H = 12_{10} = 1100_2$$

$$D_H = 13_{10} = 1101_2$$

$$E_H = 14_{10} = 1110_2$$

$$F_H = 15_{10} = 1111_2$$

1.6.3 Compattezza

Il grande vantaggio del sistema esadecimale è la compattezza dei suoi numeri, poichè la base 16 consente di utilizzare meno cifre per rappresentare un dato numero rispetto al formato binario o esadecimale

$$1111 \ 0101 \ 1100 \ 1111 \rightarrow F5CF$$

1.7 Conversione tra sistemi numerici

1.7.1 Da base $r \rightarrow$ a base 10

Consideriamo per il momento solo valori interi, la conversione da qualsiasi base r a base 10 avviene come segue

Base $r \rightarrow$ Base 10

$$N_r = d_{n-1} \cdot r^{n-1} + \dots + d_0 \cdot r^0 = M_{10}$$

N_r : Il numero di partenza espresso nella base r

M_{10} : Il numero finale ottenuto dopo la conversione, espresso in base 10

r : La radice o base del sistema numerico di partenza

n : Il numero totale di cifre che compongono il numero intero di partenza

d_i : Le singole cifre del numero di partenza, dove d_0 è la cifra meno significativa e d_{n-1} è quella più significativa

i : La posizione di ciascuna cifra, che determina la potenza della base r per cui va moltiplicata

Esempio

$$1010_2 = (1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = 10_{10}$$

$$26_8 = (2 \cdot 8^1 + 6 \cdot 8^0)_{10} = 22_{10}$$

$$431_5 = (4 \cdot 5^2 + 3 \cdot 5^1 + 1 \cdot 5^0)_{10} = 116_{10}$$

1.7.2 Da base 10 $\rightarrow$ a base r

Consideriamo per il momento solo valori interi

La conversione da base 10 a qualsiasi base r avviene come segue:

1. Dividiamo il valore numerico N per la base r fino a quando l'ultimo quoziente è minore della base stessa(r)
2. Prendiamo l'ultimo quoziente e tutti i resti della divisione, e procedendo dall'ultimo resto al primo, li scriviamo da sinistra verso destra

Esempi

- *Esempio:* Convertire il numero 12 da base 10 a base 2
- $12 : 2 = 6$ con resto = 0
- $6 : 2 = 3$ con resto = 0
- $3 : 2 = 1$ con resto = 1
- $1 : 2 = 0$ con resto = 1
- quindi:

$$12_{10} = 1100_2$$

- *Esempio:* convertire il numero 120 da Base 10 a Base 8
- $120 : 8 = 15$ con resto = 0
- $15 : 8 = 1$ con resto = 7

- $1 : 8 = 0$ con resto 1

quindi:

$$120_{10} = 170_8$$

- *Esempio*: convertire il numero 1253 da base 10 a base 16
- $1253 : 16 = 78$ con resto = 5
- $78 : 16 = 4$ con resto = 14 = E
- $4 : 16 = 0$ con resto 4

quindi:

$$1253_{10} = 4E5_{16}$$

1.7.3 Da base $r \rightarrow$ a base r'

Consideriamo per il momento solo valori interi

La conversione da qualsiasi base r a qualsiasi base r' avviene come segue:

1. Convertire da base r a base 10
2. Convertire il risultato da base 10 a base r'



Esempi

- *Esempio*: Convertire il numero $AB2_{16}$ in binario.
- $A_{16} = 10_{10} = 1010_2$
- $B_{16} = 11_{10} = 1011_2$
- $2_{16} = 2_{10} = 0010_2$
- quindi

$$AB2_{16} = 1010\ 1011\ 0010_2$$

- *Esempio*: Convertire il numero $(516)_8$ in Binario.
- $5_8 = 5_{10} = 101_2$
- $1_8 = 1_{10} = 001_2$
- $6_8 = 6_{10} = 110_2$
- quindi

$$516_8 = 101001110_2$$

- E se dovessimo convertire un numero da base 2 a base 8?

$$101011_2 = ???????_8$$

Svolgimento: raggruppiamo i bit a 3 a 3 da destra: $(101)_2$ e $(011)_2$

$$- 101_2 = 5_8$$

$$- 011_2 = 3_8$$

quindi:

$$101011_2 = 53_8$$

- Oppure da base 8 a base 2?

$$5371_8 = \text{????}_2$$

Svolgimento: convertiamo ogni cifra ottale in un gruppo di 3 bit:

$$- 5_8 = 101_2$$

$$- 3_8 = 011_2$$

$$- 7_8 = 111_2$$

$$- 1_8 = 001_2$$

quindi:

$$5371_8 = 101011111001_2$$

- E da base 8 a base 16?

$$742_8 = \text{???}_{16}$$

Svolgimento:

1. Passiamo prima in binario (3 bit per cifra ottale):

$$742_8 = \underbrace{111}_7 \underbrace{100}_4 \underbrace{010}_{2(2)} = 111100010_2$$

2. Convertiamo da binario a esadecimale raggruppando a 4 bit da destra:

$$(0001)(1110)(0010)_2$$

$$- 0001_2 = 1_{16}$$

$$- 1110_2 = 14_{10} = E_{16}$$

$$- 0010_2 = 2_{16}$$

quindi:

$$742_8 = 1E2_{16}$$

1.8 Rappresentazioni dei valori

1.8.1 Limite di cifre

La rappresentazione dei valori è legata al numero di cifre disponibili, nei sistemi di elaborazione, come in generale in tutte le applicazioni pratiche, il numero di cifre impiegate nella rappresentazioni di valori numerici è limitato.

Si ha **overflow** quando si è nell'impossibilità di rappresentare il risultato di una operazione con il numero di cifre a disposizione

1.8.2 Tabella numeri rappresentabili

n numero di bit	2^n numero di valori	$2^n - 1$ max valore rappresentabile
1	2	1
2	4	3
3	8	7
4	16	15
5	32	31
6	64	63
...	...	...

1.8.3 Prefissi

$$\text{Kilo} = 2^{10} = 1024$$

$$\text{Mega} = 2^{20} = 1048576$$

$$\text{Giga} = 2^{30} = 1073741824$$

2. Sistema binario: Operazioni aritmetiche

2.1 Somma

2.1.1 Definizione

Si esegue la somma tra i bit di pari ordine

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \quad \text{con riporto 1 sul bit di ordine superiore}$$

$$1 + 1 + 1 = 1 \quad \text{con riporto 1 sul bit di ordine superiore}$$

La somma è definita su 3 elementi:

- 2 addendi
- Il riporto (carry)

La somma di 2 unità (valore 1) di un dato ordine, creano 1 unità dell'ordine immediatamente superiore (carry)

2.1.2 Esempio

Si esegua la **somma** tra 010011 e 010001 (che indicano, rispettivamente, i numeri decimali 19 e 17)

RIPORTO	1	0	0	1	1	
PRIMO ADDENDO	0	1	0	0	1	1
SECONDO ADDENDO	0	1	0	0	0	1
SOMMA	1	0	0	1	0	0

36 in decimale

2.1.3 Esempio con Overflow

Supponendo di avere a disposizione 6 bit, si calcoli la somma e la sottrazione in binario dei seguenti numeri:

010101 e 110100

Effettuando la **somma**:

RIPORTO	1		1			
PRIMO ADDENDO	0	1	0	1	0	1
SECONDO ADDENDO	1	1	0	1	0	0
SOMMA	0	0	1	0	0	1



il numero ottenuto non è rappresentabile in quanto il risultato è su 7 bit!

2.2 Sottrazione

2.2.1 Definizione

Si esegue la differenza tra i bit di pari ordine

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1 \quad \text{con prestito dal bit di ordine immediatamente superiore}$$

La sottrazione opera su 3 gruppi di bit:

- Minuendo e Sottraendo
- Prestito proveniente dalla cifra di ordine immediatamente superiore

Ogni volta che si deve sottrarre dalla cifra 0 la cifra 1, occorre chiedere in prestito una unità alla cifra di ordine immediatamente superiore che vale due unità della cifra di ordine inferiore

2.2.2 Esempio

Si esegua la **sottrazione** tra 11101 e 01110 (che indicano, rispettivamente, i numeri decimali 29 e 14)

PRESTITO		2	2	2	
MINUENDO	1	1	1	0	1
SOTTRAENDO	0	1	1	1	0
DIFFERENZA	0	1	1	1	1

15 in decimale

2.2.3 Esempio di Sottrazione Impossibile (Underflow)

Supponendo di avere a disposizione 6 bit, si calcoli la differenza in binario tra 010101 e 110100:

PRESTITO						
MINUENDO	0	1	0	1	0	1
SOTTRAENDO	1	1	0	1	0	0
DIFFERENZA	???	0	0	0	0	1

Non è possibile eseguire l'operazione

Perché non è possibile effettuare l'operazione?

1. Per eseguire l'ultimo prestito a sinistra occorrerebbe un bit di ordine superiore: servirebbero **7 bit** anziché 6.
2. Il calcolo equivale a $(010101 - 110100)_2 = (21 - 52)_{10} = -31_{10}$. I numeri **negativi non sono rappresentabili** nel sistema binario puro (senza segno).

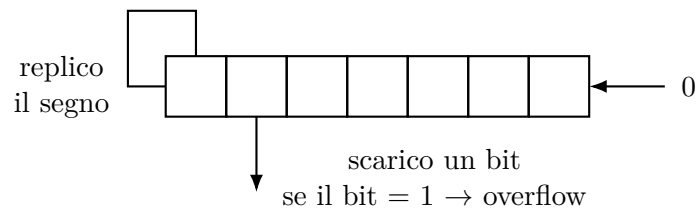
2.3 Shift

2.3.1 Definizione

Consiste nello spostare verso destra / sinistra la posizione delle cifre di un numero, espresso in una base qualsiasi, inserendo uno zero nelle posizioni lasciate libere

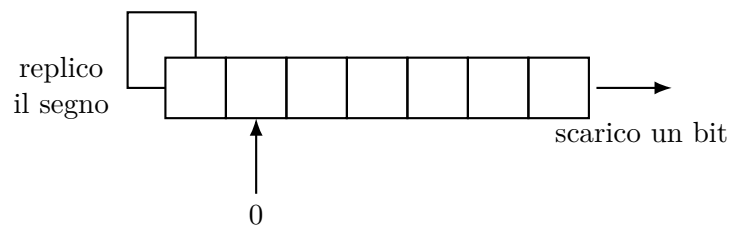
2.3.2 Left Shift

Equivale a **moltiplicare** il numero per la base



2.3.3 Right Shift

Equivale a **dividere** il numero per la base



3. Rappresentazione dei numeri negativi

3.1 Perchè ne abbiamo bisogno?

Come visto in precedenza, con la modalità di rappresentazione semplice non è possibile rappresentare numeri negativi.

Per ovviare a questo problema sono stati definiti varie modalità di rappresentazione dei numeri interi

3.2 MS - Modulo e Segno

3.2.1 Procedura

Supponiamo di avere a disposizione 1 Byte per rappresentare numeri sia positivi che negativi.

Ricorrendo al metodo del Modulo e Segno utilizzeremo:

- I primi 7 bit da destra per il valore assoluto del numero
- Il bit più a sinistra (MSB) per indicare il segno
 - 1 se il numero è negativo
 - 0 se è positivo

3.2.2 Range di rappresentabilità

Con n bit totali si possono rappresentare i numeri interi nell'intervallo

$$[-(2^{n-1} - 1), +(2^{n-1} - 1)]_{10}$$

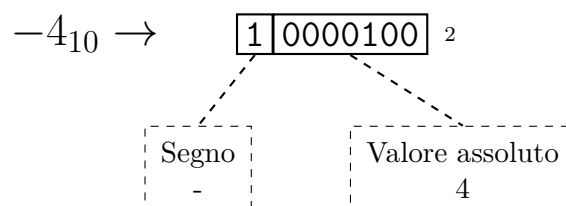
3.2.3 Difetti del sistema

- Esistono 2 diverse rappresentazioni dello 0

$$0000_2 = +0_{10} \quad 1000_2 = -0_{10}$$

- Un bit tra tutti i bit disponibili viene "speso" per il segno

3.2.4 Esempio di conversione



3.2.5 Somma

Come si esegue la somma di 2 valori in Modulo e Segno?

Confronto i bit di segno dei 2 numeri

1. Se i bit di segno sono **uguali**:

1.1. Il bit di segno risultante sarà il bit di segno dei due addendi

1.2. Eseguo la somma bit a bit dei moduli (a meno di overflow)

Esempio (segni uguali): $(-2_{10}) + (-3_{10})$ su 4 bit:

Segno	Modulo	
1	010 ₂	(-2 ₁₀)
+1	011 ₂	(-3 ₁₀)
1	101 ₂	(-5 ₁₀)

2. Se i bit di segno sono **diversi**:

2.1. Confronto i valori assoluti

2.2. Il bit di segno risultante sarà il bit di segno dell'addendo con valore assoluto maggiore

2.3. Eseguo la differenza bit a bit tra il modulo maggiore e quello minore

Esempio (segni diversi): $(+3_{10}) + (-2_{10})$ su 4 bit:

Segno	Modulo	
0	011 ₂	(+3 ₁₀)
+1	010 ₂	(-2 ₁₀)
0	001 ₂	(+1 ₁₀)

3.2.6 Sottrazione

Come si esegue la sottrazione di due valori in Modulo e Segno:

Confronto i bit di segno dei 2 numeri:

1. Se i bit di segno sono **uguali**:

1.1. Il bit di segno risultante sarà uguale al bit di segno dell'operando a modulo maggiore

1.2. Il risultato avrà modulo pari alla differenza dei moduli degli operandi (modulo maggiore - modulo minore)

Esempio (segni uguali): $(+2_{10}) - (+3_{10})$ su 4 bit:

Segno	Modulo	
0	010 ₂	(+2 ₁₀ , minuendo)
-0	011 ₂	(+3 ₁₀ , sottraendo)
1	001 ₂	(-1 ₁₀ , poiché 3 > 2 prende il segno del sottraendo)

2. Se i bit di segno sono **diversi**:

2.1. Il bit di segno risultante sarà uguale al bit di segno del minuendo

2.2. Il risultato avrà modulo pari alla somma dei moduli dei 2 operandi

Esempio (segni diversi): $(+2_{10}) - (-3_{10})$ su 4 bit:

Segno	Modulo	
0	010_2	$(+2_{10}, \text{ minuendo})$
-1	011_2	$(-3_{10}, \text{ sottraendo})$
0	101_2	$(+5_{10}, \text{ prende il segno del minuendo e somma i moduli})$

3.3 CA1 - Complemento a 1

3.3.1 Complemento

Come indica il nome stesso, questo metodo si basa sull'operazione di **complemento**.

Con **complemento** si intende l'operazione che associa ad un bit il suo opposto, cioè il valore ottenuto sostituendo gli 1 con 0, e tutti gli 0 con 1

3.3.2 Procedura

Il metodo CA1 è molto semplice e diretto:

1. Se il numero da codificare è **positivo** lo si converte con il metodo binario tradizionale
2. Se il numero da codificare è **negativo** basta convertire in binario il suo modulo e quindi eseguire l'operazione di complemento sulla codifica binaria effettuata

3.3.3 Range di numeri rappresentabili

Con n bit totali si possono rappresentare i numeri interi nell'intervallo

$$[-(2^{n-1} - 1), +(2^{n-1} - 1)]_{10}$$

3.3.4 Difetti

Anche in questo caso sussiste il problema delle due diverse rappresentazioni dell'0

$$0000\ 0000 (+0)$$

$$1111\ 1111 (-0)$$

3.3.5 Esempi di conversione

- supponiamo di avere 4 bit e di voler rappresentare in CA1 il valore 3_{10}

$$3_{10} = 0011_2$$

- supponiamo di avere 4 bit e di voler rappresentare in CA1 il valore -3_{10}

$$-3_{10} = \overline{0011}_2 = 1100_2$$

3.4 CA2 - Complemento a 2

3.4.1 Procedura

1. Se il numero X è **positivo** esso rimane invariato
2. Se il numero X è **negativo**
 - 2.1. Si effettua il CA1 sul valore da codificare
 - 2.2. Si somma +1 al risultato ottenuto con CA1

In CA2 i valori negativi hanno $MSB = 1$

3.4.2 Conversione a decimale

- Se il numero è **positivo** ($MSB = 0$), si converte in base decimale usando il numero binario puro
- Se il numero è **negativo** ($MSB = 1$), si applica l'operazione di CA2 a questo valore ottenendo la rappresentazione del corrispondente positivo, si converte il risultato come numero in binario puro e si aggiunge il segno meno

3.4.3 Range di numeri rappresentabili

Il metodo CA2 super il principale difetto di CA1, ossia la presenza di una doppia codifica per lo 0.

Dati n bit, si possono rappresentare i numeri interi nell'intervallo

$$[-(2^{n-1}), +(2^{n-1} - 1)]$$

3.4.4 Esempi di conversione

Bit	Valore Assoluto	Complemento a 2
0111 1111	127	127
0111 1110	126	126
0000 0010	2	2
0000 0001	1	1
0000 0000	0	0
1111 1111	255	-1
1111 1110	254	-2
1000 0010	130	-126
1000 0001	129	-127
1000 0000	128	-128

3.4.5 Somma

Come si esegue la somma di 2 valori in CA2?

1. Si esegue la somma su tutti i bit degli addendi, compreso il segno
2. Un eventuale riporto oltre il bit di segno viene scartato

3. Nel caso di operandi di segno concorde occorre verificare la presenza o meno di overflow

Esempio 1

$$3 + (-8)$$

$$\begin{array}{r} (+3) \quad 0000 \ 0011 \\ +(-8) \quad 1111 \ 1000 \\ \hline (-5) \quad 1111 \ 1011 \end{array}$$

Esempio 2

$$-2 + (-5)$$

$$\begin{array}{r} (-2) \quad 1111 \ 1110 \\ +(-5) \quad 1111 \ 1011 \\ \hline (-7) \quad 1 \ 1111 \ 1001 \quad : \text{scarto il carry} \end{array}$$

3.4.6 Sottrazione

La sottrazione tra 2 numeri in CA2 viene trasformata in somma applicando la regola

$$A - B = A + (-B)$$

ovvero

$$A - B = A + CA2(B)$$

Per assicurarsi della correttezza del risultato di un'operazione di somma in CA2 bisogna verificare l'assenza di overflow

- Non si ha overflow se gli operandi hanno segno discorde
- Si ha overflow se gli operandi hanno segno concorde e il segno del risultato è discorde con essi

3.5 Rappresentazione eccesso 2^{n-1}

3.5.1 Procedure

Nella rappresentazione eccesso 2^{n-1} un numero X è rappresentato come segue:

$$X + 2^{n-1}$$

Regola pratica

I numeri in eccesso 2^{n-1} si ottengono da quelli in CA2 complementando il bit più significativo

3.5.2 Range numeri rappresentabili

$$[-(2^{n-1}), +(2^{n-1} - 1)]$$

3.5.3 Esempio di conversione

Vogliamo rappresentare il valore $X = -3$ su $n = 4$ bit.

Metodo 1: Applicazione della formula $X + 2^{n-1}$

1. Calcoliamo il valore da codificare:

$$-3 + 2^3 = -3 + 8 = 5_{10}$$

2. Convertiamo il valore 5_{10} ottenuto in binario puro a 4 bit:

$$5_{10} = 0101_2$$

Metodo 2: Tramite la Regola Pratica (da CA2)

1. Rappresentiamo $X = -3$ in Complemento a 2 (CA2) su 4 bit:

$$-3_{10} \xrightarrow{\text{CA2}} 1101_2$$

2. Complementiamo (invertiamo) il bit più significativo (MSB):

$$1101_2 \xrightarrow{\text{inverti MSB}} 0101_2$$

3.6 Rappresentazione Eccesso 128

3.6.1 Procedura

Nella rappresentazione Eccesso 128 un numero X è rappresentato come segue:

$$X + 128$$

3.6.2 Esempi

$$\begin{array}{lll} X = 5 & 5 + 128 = 133 & 1000\ 0101_2 \\ X = -3 & -3 + 128 = 125 & 0111\ 1101_2 \end{array}$$

Passando da CA2:

$$-3_{10} \longrightarrow \overbrace{00000011_2}^3 \longrightarrow \overbrace{11111100_{CA2}}^{*3} \xrightarrow{CA2} \overbrace{11111101_{CA2}}^{-3} \xrightarrow{\text{complemento il bit più significativo}} \overbrace{01111101_{Eccesso128}}^{-3}$$

3.6.3 Osservazioni

- In Eccesso 128 i numeri da $[-128, 127]$ si mappano su $[0, 255]$
- In Eccesso, usa una convenzione per il bit di segno opposta: 0 per i numeri negativi e 1 per quelli positivi.

4. Rappresentazione numeri frazionari con la virgola fissa

4.1 Idea

Un sistema di numerazione in virgola fissa è quello in cui:

- Si riserva un numero fisso di bit per parte intera e parte frazionaria
- La posizione della virgola decimale è implicita
- La posizione della virgola decimale è uguale in tutti i numeri

4.2 UNSIGNED fixed point

4.2.1 Cosa è?

La rappresentazione in virgola fissa non segnalata (Unsigned Fixec-Point) è una tecnica utilizzata per codificare e memorizzare numeri reali positivi (o nulli) all'interno di un numero finito di bit, mantenendo una posizione fissa per la virgola decimale

4.2.2 Struttura del formato

A differenza dei numeri interi, dove tutti i bit rappresentano potenze intere di 2, nella virgola fissa il totale di bit a disposizione (N) viene suddiviso in 2 parti distinte da una posizione virtuale della virgola:

- **Parte Intera (I bit)**: Rappresenta la componente intera del numero
- **Parte frazionaria / decimale (D bit)**: Rappresenta la parte dopo la virgola

Il totale dei bit sarà $N = I + D$, non essendo presente alcun bit per il segno, tutti i bit sono destinati al valore del numero

I-1	I-2	...	0	-1	-2	...	-D
Parte Intera				Parte Frazionaria			

4.2.3 Range di valori rappresentabili

- Con questo metodo l'**intervallo di numeri interi** rappresentabili è: $[0, 2^{I-1}]$
- L'intervallo rappresentabile dalla **parte decimale** è $[0, 2^{D-1}]$

4.2.4 Peso delle posizioni

Ogni bit ha un peso associato ad una potenza di 2, a seconda della sua posizione rispetto alla virgola:

$$b_{I-1} \cdot \dots \cdot b_1 \cdot b_0 \cdot b_{-1} \cdot \dots \cdot b_{-D}$$

- **Parte Intera**

- $b_0 \rightarrow 2^0 = 1$

- $b_1 \rightarrow 2^1 = 2$

- $b_{I-1} \rightarrow 2^{I-1}$

- **Parte Frazionaria**

- $b_{-1} \rightarrow 2^{-1} = 0.5$

- $b_{-2} \rightarrow 2^{-2} = 0.25$

- $b_{-3} \rightarrow 2^{-3} = 0.125$

- $b_{-D} \rightarrow 2^{-D}$

Il valore decimale V corrispondente a una sequenza di bit si calcola sommando i pesi dei bit pari a 1:

$$V = \sum_{k=-D}^{I-1} b_k \cdot 2^k$$

4.2.5 Da Decimale a Virgola Fissa Unsigned

Prendiamo come esempio la conversione del numero decimale 23.625_{10} utilizzando un formato a 16 bit complessivi, con 8 bit per la parte intera ($I = 8$) e 8 bit per la parte frazionaria ($D = 8$)

1. Si converte la parte intera in binario standard

$$23_{10} = 16 + 4 + 2 + 1 = 2^4 + 2^2 + 2^1 + 2^0 = 0001\ 0111_2$$

2. Si moltiplica ripetutamente per 2 la parte frazionaria, prendendo la parte intera del risultato come bit e proseguendo con la nuova parte frazionaria fino a ottenere 0

- (a) $0.625 \times 2 = 1.25 \rightarrow 1$ (primo bit a destra della virgola)

- (b) $0.25 \times 2 = 0.50 \rightarrow 0$ (secondo bit)

- (c) $0.50 \times 2 = 1.00 \rightarrow 1$ (terzo bit, stop perchè la parte decimale è zero)

Il risultato finale sarà:

- $I = 0001\ 0111$
- $D = 101$
- Risultato finale = $0001\ 0111.10100000$

4.2.6 Da Virgola Fissa Unsigned a Decimale

Prendiamo la stringa binaria

00010111.10100000₂

e calcoliamone il valore sommando i pesi delle posizioni con bit 1:

$$\begin{aligned} V &= (1 \cdot 2^4) + (1 \cdot 2^2) + (1 \cdot 2^1) + (1 \cdot 2^0) + (1 \cdot 2^{-1}) + (1 \cdot 2^{-3}) \\ &= 16 + 4 + 2 + 1 + 0.5 + 0.125 \\ &= \mathbf{23.625}_{10} \end{aligned}$$

4.3 SIGNED fixed point

4.3.1 Struttura del formato

L'unica differenza concettuale rispetto all'Unsigned è che il bit più significativo (MSB), ovvero il primo bit a sinistra, assume valore di peso negativo

Data una lunghezza totale di $N = I + D$ bit:

- 1 bit per il segno / parte intera più significativa
- $I - 1$ bit per il resto della parte intera
- D bit per la parte frazionaria

4.3.2 Peso delle posizioni

$$b_{I-1} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{-D}$$

- **Bit di segno (MSB)**

$$- b_{I-1} \rightarrow -2^{I-1}$$

- **Resto della parte intera**

$$- b_1 \rightarrow 2^1 = 2$$

$$- b_0 \rightarrow 2^0 = 1$$

- **Parte Frazionaria**

$$- b_{-1} \rightarrow 2^{-1} = 0.5$$

$$- b_{-2} \rightarrow 2^{-2} = 0.25$$

$$- b_{-D} \rightarrow 2^{-D}$$

Il valore decimale V si calcola con la formula del complemento a 2 estesa ai decimali:

$$V = -b_{I-1} \cdot 2^{I-1} + \sum_{k=-D}^{I-2} b_k \cdot 2^k$$

4.3.3 Da numero decimale positivo a Signed Fixed-Point

I numeri positivi si codificano esattamente come nel caso unsigned, assicurandosi solo che il MSB sia 0

1. **Parte intera:** $23_{10} = 0001\ 0111_2$
2. **Parte frazionaria:** $0.625 \times 2 \dots \rightarrow .10100000_2$
3. **Risultato:** 00010111.10100000_2

4.3.4 Da numero decimale negativo a Signed Fixed-Point

Per codificare un numero negativo in CA2 a virgola fissa, il metodo semplice è complementare a 2 la rappresentazione del corrispondente numero positivo

1. Prendi la stringa del valore positivo: 00010111.10100000
2. Invertire tutti i bit: 11101000.01011111
3. Aggiungi 1 al bit meno significativo (LSB), cioè all'ultimo bit della parte frazionaria:
 $11101000.01011111 + 00000000.00000001$
4. Risultato finale per -23.625_{10} : **11101000.01100000₂**

5. Rappresentazione numeri frazionari con la virgola mobile

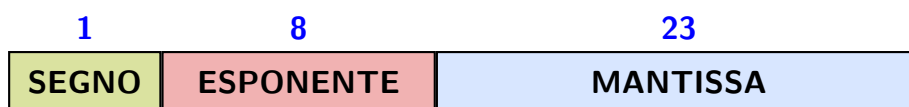
5.1 Cosa è?

La rappresentazione in virgola mobile è il metodo utilizzato dagli elaboratori per memorizzare e calcolare i numeri reali o numeri con ordini di grandezza estremamente grandi o piccoli.

5.2 Struttura

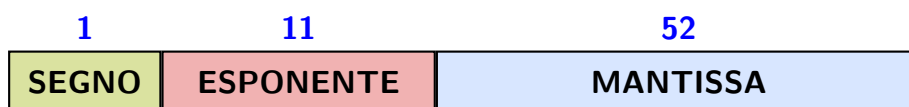
5.2.1 32 bit

I 32 bit totali sono suddivisi nel seguente modo:



5.2.2 64 bit

I 64 bit totali sono suddivisi nel seguente modo per la modalità a doppia precisione:



5.3 Formule

5.3.1 Formula della rappresentazione in virgola mobile

$$N = (-1)^S \pm M \cdot B^{\pm E}$$

- S - Segno: Determina il segno del numero
- M - Mantissa: È la parte frazionaria del numero.
Contiene le cifre effettive del valore, di norma viene normalizzata
- B - Base: È la base del sistema di numerazione utilizzato
- E - Esponente: È l'esponente a cui viene elevata la base.
Determina la posizione della virgola, ovvero l'ordine di grandezza del numero

5.3.2 Normalizzazione e "1 implicito"

Per ottimizzare lo spazio di memoria, prima di codificare il numero in binario, lo si normalizza.

Un numero binario è formalizzato quando viene scritto nella forma

$$\pm 1.M \cdot 2^E$$

- M - Mantissa: È la parte frazionaria del numero.
Contiene le cifre effettive del valore, di norma viene normalizzata
- E - Esponente: È l'esponente a cui viene elevata la base.
Determina la posizione della virgola, ovvero l'ordine di grandezza del numero

5.4 Conversione da Decimale a Virgola Mobile

Convertiamo il numero 13.25 in virgola mobile a 32 bit

1. **Determinare il segno:** Il numero è positivo, quindi il bit del segno (S) è 0

2. **Convertire la parte intera e frazionaria in binario**

(a) **Parte intera:** $31 = 1101$

(b) **Parte frazionaria:** Si moltiplica ripetutamente per 2 la parte decimale:

i. $0.25 \cdot 2 = 0.5 \rightarrow 0$

ii. $0.50 \cdot 2 = 1.0 \rightarrow 1$

Otteniamo la forma binaria: 1101.01_2

3. **Normalizzare il valore:** Spostiamo la virgola a sinistra di 3 posizioni per ottenere la forma $1.xxx$

$$1101.01_2 = 1.10101_2 \cdot 2^3$$

- L'esponente reale (E) è 3
- La parte a destra della virgola (Mantissa - M) è 10101

4. **Calcolare l'esponente in Eccesso 127:** Aggiungiamo la costante di offset all'esponente reale

$$E = 3 + 127 = 130_{10}$$

Convertendo 130 su 8 bit in binario otteniamo: 10000010

5. **Assemblare la sequenza di 32 bit**

- Segno (1 bit) = 0
- Esponente (8 bit) = 10000010
- Mantissa (23 bit) = 10101 allungato con zeri a destra fino a completare i 23 bit:

10101000000000000000000

Risultato finale:

0 10000010 10101000000000000000000

5.5 Conversione da Virgola Mobile a Decimale

Data la sequenza 1 10000000 1100000000000000000000

1. **Segno:** Il primo bit è 1, dunque il numero è negativo

2. **Esponente:**

$$10000000_2 = 128_{10}$$

Sottraendo l'offset 127: $e = 128 - 127 = 1$

3. **Mantissa:** 1100000000000000000000 rappresenta

$$2^{-1} + 2^{-2} = 0.5 + 0.25 = 0.75$$

4. **Ricostruzione del valore**

$$\text{Valore} = -(1 + \text{Mantissa}) \times 2^e = -(1 + 0.75) \times 2^1 = -1.75 \times 2 = -3.5_{10}$$

5.6 Errore assoluto E_a

5.6.1 Cosa è?

L'errore assoluto indica la differenza diretta tra il valore reale esatto e il valore approssimato memorizzato.

Rappresenta lo scarto fisico o la distanza quantitativa tra la misura reale e quella approssimata, mantenendo la stessa unità di misura del dato di partenza

5.6.2 Formula

$$E_a = |V - \tilde{V}|$$

- V : Valore reale
- $\tilde{V}$: Valore approssimato memorizzato

5.6.3 Esempio di calcolo

Dati di partenza

Ipotizziamo di dover rappresentare una grandezza fisica o un numero reale in un calcolatore:

- **Valore reale esatto (V):** 14.888
- **Valore approssimante memorizzato ($\tilde{V}$):** 14.900

Calcolo

$$E_a = |14.888 - 14.9000| = |-0.012| = 0.012$$

5.7 Errore Relativo E_r

5.7.1 Cos'è?

L'errore relativo misura l'incidenza dell'errore assoluto rispetto alla grandezza del numero di partenza.

Un errore assoluto di 0.1 è enorme se si sta misurando un oggetto lungo 1, ma è del tutto trascurabile se si sta misurando una distanza di 1000000.

Per questo motivo, l'errore relativo è un numero adimensionale ed è fondamentale per valutare la precisione di una rappresentazione

5.7.2 Formula

$$E_r = \frac{E_a}{|V|} = \frac{|V - \tilde{V}|}{|V|}$$

- E_a : Errore assoluto
- V : Valore reale
- $\tilde{V}$: Valore approssimato memorizzato

5.7.3 Esempio di calcolo

Supponiamo di voler rappresentare il valore reale $V = 14.888$ tramite un numero discreto con un passo di discretizzazione, dove il valore approssimante più vicino è $\tilde{V} = 14.9$:

1. Calcolo dell'errore assoluto

$$E_a = |14.888 - 14.9| = |-0.012| = 0.012$$

2. Calcolo dell'errore relativo

$$E_r = \frac{0.012}{14.888} \approx 0.00080602$$

3. Calcolo dell'errore relativo percentuale

$$E_r\% = 0.00080602 \times 100\% \approx 0.0806\%$$

6. Rappresentazione caratteri

6.1 Idea

Possiamo associare a ogni carattere un numero.

I caratteri possono essere rappresentati in:

- **ASCII standard:** 1 carattere è rappresentato con 7 bit per un totale di 128 simboli rappresentabili
- **ASCII estesa:** 1 carattere è rappresentato con 8 bit rappresentabili fino a 256 simboli
- **UNICODE:** 1 carattere è rappresentato con un numero maggiore di bit (tra 8 e 32 bit per carattere)

6.2 ASCII Standard

6.2.1 Caratteristiche

- ASCII standard contiene:
 - 26 + 26 lettere (maiuscole + minuscole)
 - 10 cifre decimali
 - segni di interpunzione
 - caratteri di controllo
- Le cifre sono ordinate per valore
- Le lettere maiuscolo sono ordinate alfabeticamente
- Le lettere minuscole sono ordinate alfabeticamente

6.2.2 Tabella

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	32	20	SPC	64	40	@	96	60	‘
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	\$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	,	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

6.3 ASCII Esteso

6.3.1 Caratteristiche

Uguale a l'ASCII standard ma con più caratteri

6.3.2 Tabella

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	–	160	A0	NBSP	192	C0	À	224	E0	à
129	81	–	161	A1	ı	193	C1	Á	225	E1	á
130	82	–	162	A2	¢	194	C2	Â	226	E2	â
131	83	–	163	A3	£	195	C3	Ã	227	E3	ã
132	84	–	164	A4	¤	196	C4	Ä	228	E4	ä
133	85	–	165	A5	¥	197	C5	Å	229	E5	å
134	86	–	166	A6	¦	198	C6	Æ	230	E6	æ
135	87	–	167	A7	§	199	C7	Ç	231	E7	ç
136	88	–	168	A8	¨	200	C8	È	232	E8	è
137	89	–	169	A9	©	201	C9	É	233	E9	é
138	8A	–	170	AA	ª	202	CA	Ê	234	EA	ê
139	8B	–	171	AB	«	203	CB	Ë	235	EB	ë
140	8C	–	172	AC	¬	204	CC	Ì	236	EC	ì
141	8D	–	173	AD	SHY	205	CD	Í	237	ED	í
142	8E	–	174	AE	®	206	CE	Î	238	EE	î
143	8F	–	175	AF	—	207	CF	Ï	239	EF	ï
144	90	–	176	B0	°	208	D0	Ð	240	F0	ð
145	91	–	177	B1	±	209	D1	Ñ	241	F1	ñ
146	92	–	178	B2	²	210	D2	Ò	242	F2	ò
147	93	–	179	B3	³	211	D3	Ó	243	F3	ó
148	94	–	180	B4	´	212	D4	Ô	244	F4	ô
149	95	–	181	B5	µ	213	D5	Õ	245	F5	õ
150	96	–	182	B6	¶	214	D6	Ö	246	F6	ö
151	97	–	183	B7	·	215	D7	×	247	F7	÷
152	98	–	184	B8	¸	216	D8	Ø	248	F8	ø
153	99	–	185	B9	¹	217	D9	Ù	249	F9	ù
154	9A	–	186	BA	º	218	DA	Ú	250	FA	ú
155	9B	–	187	BB	»	219	DB	Û	251	FB	û
156	9C	–	188	BC	¼	220	DC	Ü	252	FC	ü
157	9D	–	189	BD	½	221	DD	Ý	253	FD	ý
158	9E	–	190	BE	¾	222	DE	Þ	254	FE	þ
159	9F	–	191	BF	¿	223	DF	ß	255	FF	ÿ

6.4 UNICODE

6.4.1 Caratteristiche

- È una evoluzione dello standard ASCII
- UNICODE è uno standard per la rappresentazione di caratteri
- Codifica tutti i caratteri utilizzati nelle principali lingue del mondo
- Indipendente dalla lingua, dal sistema operativo e dal programma utilizzato
- Inizialmente rappresentato come una codifica su 16 bit, ma poi esteso a 24 e 32 bit
- Unicode è in continua evoluzione e continua ad aggiungere sempre più caratteri
- Un carattere UNICODE è caratterizzato dal suo codice numerico, detto code point, solitamente rappresentato con 8 cifre esadecimali
- Con UNICODE, è possibile creare e gestire senza troppa pena documenti multilingue
- In particolare, tuttigli gli standard W3C supportano UNICODE

7. Instruction Set Architecture (ISA)

7.1 Cosa è?

È la struttura attraverso cui tutte le operazioni vengono codificati.

Noi prenderemo d'esempio l'architettura MIPS

7.2 Architettura MIPS

7.2.1 Caratteristiche

- Architettura di tipo RISC: semplice, uniforme e facile da capire
- 32 registri di 32 bit
- Permette:
 - Manipolazione di dati solo sui registri
 - Trasferimento di dati tra memoria e registri
 - Alterazione del flusso di controllo ("Salti" condizionati e non)

7.3 Formati delle istruzioni

- **R-Type**: Operazioni tra 2 registri
- **I-Type**: Operazioni immediate tra un registro e un valore "fisso"
- **J-Type**: Operazioni di salto condizionato e non

7.4 Rappresentazioni disponibili

7.4.1 Stringa a 32 bit

00000001000010010101000000100000

Una semplice stringa di 32 bit consecutivi

7.4.2 Stringa a 32 bit sezionata

000000 01000 01001 01010 00000 100000

Una semplice stringa di 32 bit consecutivi sezionata con degli spazi per indicare ogni campo dell'istruzione

7.4.3 Rappresentazione esadecimale

0x 1095020

Una semplice stringa di 32 bit consecutivi sezionata con degli spazi per indicare ogni campo dell'istruzione

7.4.4 Formato assembly

add \$10, \$8, \$9

- Un formato un pò più comodo:
 - codici operativi simbolici
 - nomi simbolici di registri
 - nomi simbolici di variabili
- I registri si trovano a pagina 24 dell'appendice A
- Gli opcode/funct si trovano a pagina 50 dell'appendice A

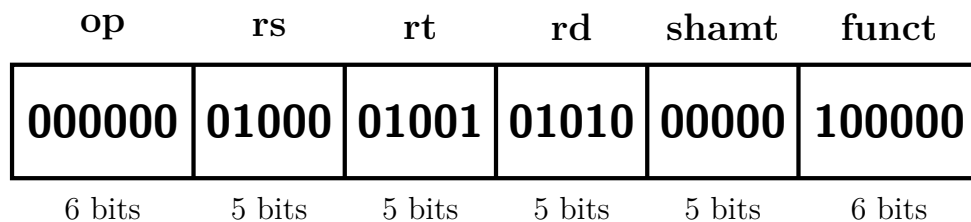
8. ISA R-Type

8.1 Cosa sono?

Sono tutte quelle istruzioni che coinvolgono l'utilizzo di 3 registri, due registri sorgente e un registro di destinazione al fine di compiere un'operazione aritmetico-logica

8.2 Struttura

8.2.1 Struttura della stringa a 32 bit



8.2.2 Spiegazione dei campi

- **opcode (6 bit)**: Indica il tipo d'istruzione ISA (R-Type, I-Type, J-Type), per le R-type è sempre 0
- **rs (5 bit)**: Il primo registro sorgente coinvolto nell'operazione
- **rt (5 bit)**: Il secondo registro sorgente coinvolto nell'operazione
- **rd (5 bit)**: Il registro di destinazione della operazione, può essere uno dei 2 registri sorgenti oppure un terzo registro differente
- **shamt (5 bit)**: Indica di quanti bit scorrere il contenuto sorgente durante le operazioni di shift
- **funct (6bit)**: Indica l'operazione da effettuare

9. ISA I-Type

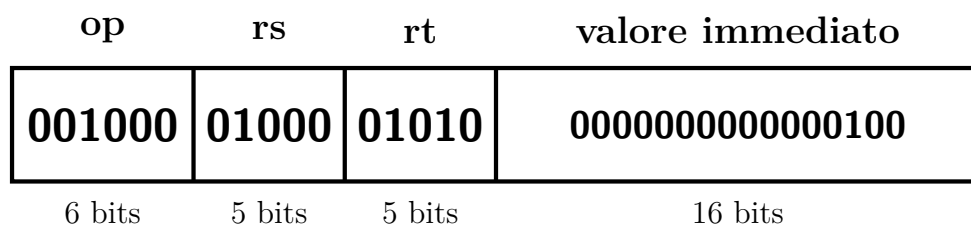
9.1 Cosa sono?

Sono tutte le istruzioni che coinvolgono l'utilizzo di 2 registri e un valore immediato.

Utilizzato per operazioni aritmetico logiche immediate e operazioni di scrittura e lettura in memoria

9.2 Struttura

9.2.1 Struttura della stringa a 32 bit

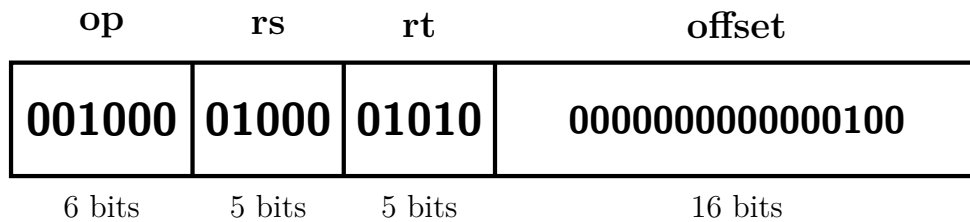


9.2.2 Spiegazione dei campi

- **opcode (6 bit)**: Indica quale operazione eseguire
- **rs (5 bit)**: Primo registro (sorgente)
- **rt (5 bit)**: Secondo registro (destinazione)
- **valore immediato (16 bit)**: Il valore immediato con cui operare

9.3 Istruzione load word

9.3.1 Struttura stringa a 32 bit



9.3.2 Spiegazione dei campi

- **opcode(6 bit)**: Per questa operazione è pari a $35_{(10)}$
- **rs (5 bit)**: Registro base
- **rt (5 bit)**: Registro destinazione
- **offset (16 bit)**: È l'offset usato per calcolare l'indirizzo a cui accedere

9.3.3 Svolgimento dell'operazione

1. Funzione generale dell'istruzione

L'istruzione `lw` serve a copiare il contenuto di una parola di memoria trasferendolo direttamente all'interno di un registro del processore

2. Uso del registro base

Il campo `rs` contiene l'indirizzo di memoria base a partire dal quale calcolare l'accesso.

Il valore memorizzato in questo registro non viene modificato dall'esecuzione dell'istruzione

3. Calcolo dell'indirizzo effettivo

L'indirizzo esatto della parola da leggere in memoria viene calcolato sommando il contenuto del registro base con l'offset a 16 bit, secondo la formula: $\text{Indirizzo} = \text{base} + \text{offset}$

4. Uso del registro destinazione

A differenza del formato R-Type in cui `rt` fa da sorgente, nella `lw` il campo `rt` funge da registro di destinazione.

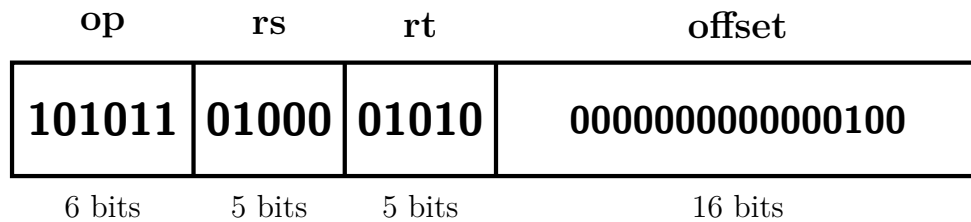
I 32 bit letti a partire dall'indirizzo di memoria calcolato vengono scritti e salvati all'interno di `rt`

5. Sintassi Assembly di riferimento

La sintassi standard è `lw $rt, offset($rs)`, carica in `$10` il valore contenuto nella cella di memoria all'indirizzo ottenuto sommando `$8` e l'offset immediato `4`

9.4 Istruzione store word

9.4.1 Struttura stringa a 32 bit



9.4.2 Spiegazione dei campi

- **opcode (6 bit):** Per questa operazione è pari a $43_{(10)}$
- **rs (5 bit):** Registro base
- **rt (5 bit):** Registro sorgente
- **offset (16 bit):** È l'offset usato per calcolare l'indirizzo a cui accedere

9.4.3 Svolgimento dell'operazione

1. Funzione generale dell'istruzione

L'istruzione **sw** effettua l'operazione inversa rispetto alla **lw**: copia il contenuto di un registro del processore e lo memorizza all'interno di una parola della memoria

2. Uso del registro sorgente

Il campo **rt** opera come registro sorgente.

I 32 bit contenuti in **rt** costituiscono il dato da trasferire e scrivere in memoria

3. Uso del registro base

Il campo **rs** serve da registro base e custodisce l'indirizzo di memoria di partenza.

Il valore contenuto in questo registro non viene modificato dall'istruzione

4. Calcolo dell'indirizzo di memoria effettivo

La posizione esatta di memoria in cui scrivere la parola viene calcolata sommando il contenuto del registro base al valore immediato di spiazzamento a 16 bit: $\text{Indirizzo} = \text{base} + \text{offset}$.

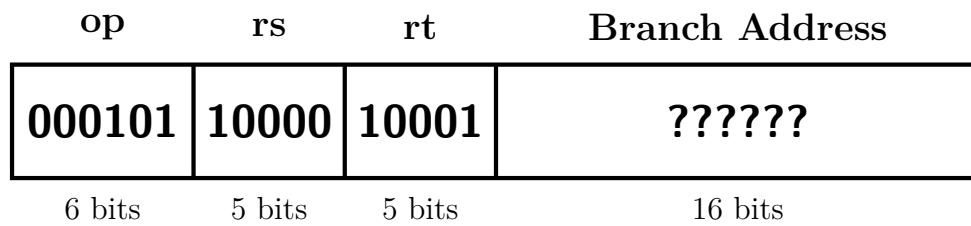
Poichè si memorizza una word completa, l'indirizzo ottenuto deve essere allineato ai multipli di 4

5. Sintassi Assembly di riferimento

La sintassi è **sw \$rt, offset(\$rs)**, ad esempio, **sw \$10, 4(\$8)** prende il dato memorizzato nel registro \$10 e lo scrive nella locazione di memoria situata all'indirizzo dato dalla somma di \$8 e dell'offset 4

9.5 Istruzioni di branch

9.5.1 Struttura stringa a 32 bit



9.5.2 Spiegazione dei campi

- opcode (6 bit)
- rs (5 bit)
- rt (5 bit)
- Branch address (16 bit)

9.5.3 Spiegazione dell'operazione

1. Decisione dell'operazione di confronto

La ALU calcola la differenza tra il contenuto di rs e rt per testare la condizione stabilita dall'opcode.

2. Eventuale salto

Se la condizione è verificata, l'indirizzo della prossima istruzione diventa $PC + 4 + (\text{Offset} \times 4)$; in caso contrario, il programma prosegue sequenzialmente all'istruzione successiva ($PC + 4$).

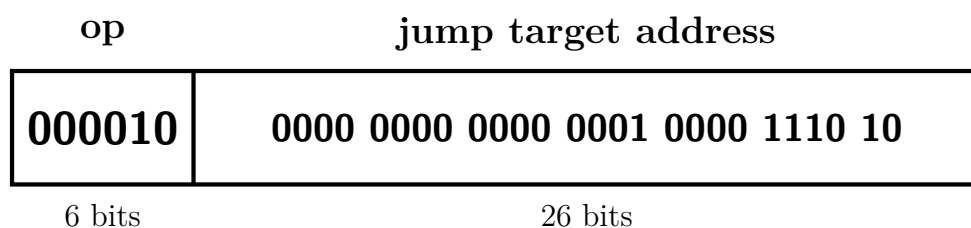
10. ISA J-Type

10.1 Cosa sono?

Sono le istruzioni di salto incondizionato, che modificano il normale flusso sequenziale del programma caricando direttamente nel Program Counter un nuovo indirizzo di memoria verso cui proseguire l'esecuzione, senza verificare alcuna condizione.

10.2 Struttura

10.2.1 Struttura della stringa a 32 bit



10.2.2 Spiegazione dei campi

- **opcode (6 bit):** In questo caso è pari a $2_{(10)}$
- **jump target address (26 bit):** Indirizzo di parola a cui saltare.

Poiché ogni istruzione è allineata a 4 byte (termina con 00), i 26 bit specificano l'indirizzo diviso per 4.

All'esecuzione, l'indirizzo a 32 bit del PC viene formato affiancando i 4 bit superiori del Program Counter corrente, i 26 bit del target e due zeri finali.

11. Assembly

11.1 Linguaggi di alto e basso livello

11.1.1 Linguaggio assembler

Cosa è?

È una rappresentazione simbolica del linguaggio macchina rendendolo così molto più comprensibile.

Viene tradotto dall'assemblatore in linguaggio macchina

Vantaggi

La dipendenza dall'architettura del calcolatore permette:

- Ottimizzare le prestazioni (maggiore efficienza)
- Programmi (potenzialmente) più compatti
- Massimo sfruttamento delle potenzialità dell'hardware sottostante
- Molto importante per programmare controller di processi e macchinari, o per apparati limitati

Svantaggi

- Minore espressività
- Necessario conoscere i dettagli dell'architettura
- Mancanza di portabilità su architetture diverse
- Difficoltà di comprensione
- Lunghezza maggiore dei programmi

11.1.2 Linguaggi ad alto livello

Cosa sono?

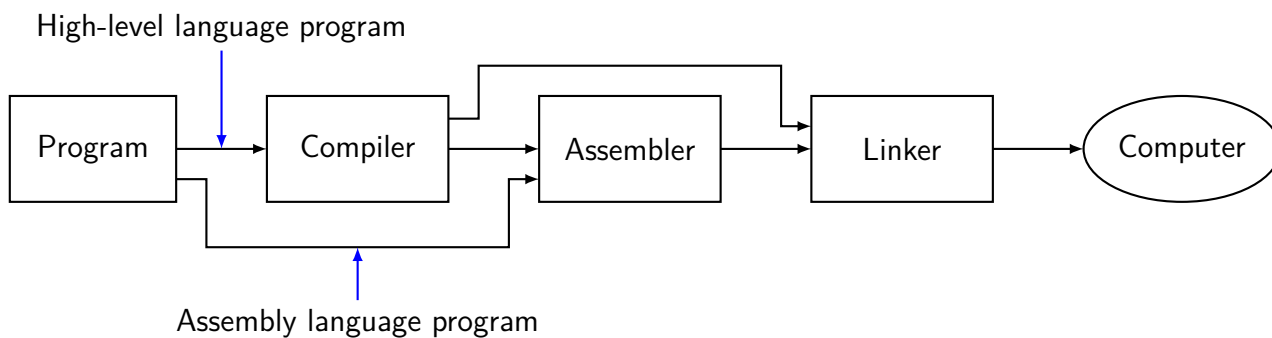
Sono linguaggi facilmente utilizzabili per gli umani, vengono compilati (o interpretati) per poi essere tradotti in assembler

Vantaggi

- Notazione vicina al linguaggio corrente e alta notazione algebrica (maggiore espressività e leggibilità)
- Incremento della produttività, la programmazione è svincolata dalla conoscenza dei dettagli architetturali della macchina utilizzata
- Indipendenza dalle caratteristiche dell'architettura su cui il programma va eseguito
- Permettono l'uso di librerie di funzionalità già scritte

11.2 Catena programmatica (Vista ad alto livello)

11.2.1 Struttura

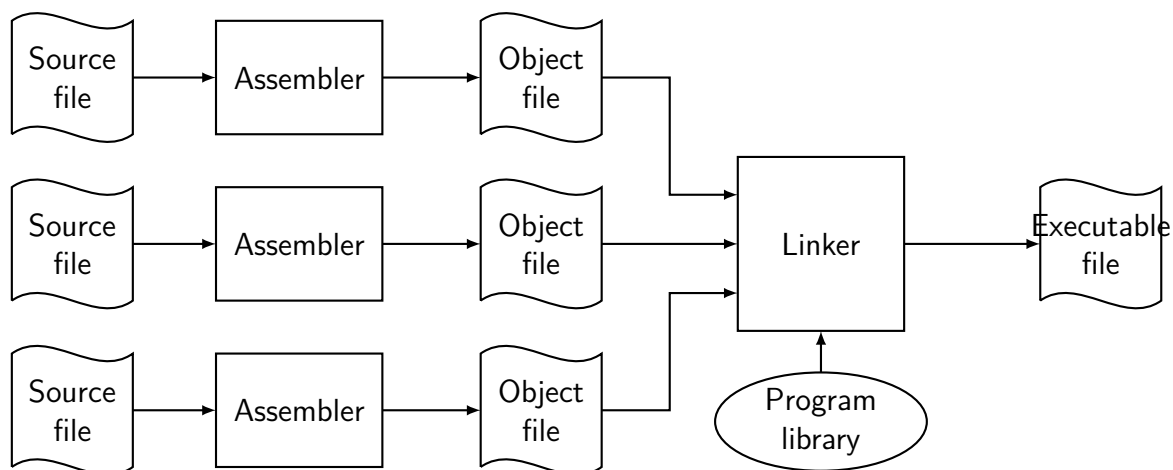


11.2.2 Ruolo del Compiler

- Un programma ad alto livello viene tradotto nel linguaggio assembler utilizzando un apposito programma: il compilatore.
- Dopo la fase di compilazione, il programma scritto in linguaggio assembler viene tradotto in linguaggio macchina da un apposito programma: l'assemblatore

11.3 Catena programmatica (Vista a basso livello)

11.3.1 Struttura



11.3.2 Assembler

Cosa fa?

- Converte un programma assembler (file sorgente) in linguaggio macchina (file oggetto)
- Gestisce le etichette
- Gestisce le pseudoistruzioni
- Gestisce numeri in base diverse

Processo di assemblaggio

L'assemblaggio è un procedimento sequenziale che esamina, riga per riga, il codice sorgente Assembly, traducendo ciascuna riga in un'istruzione del linguaggio macchina.

Questo processo di assemblaggio viene applicato modulo per modulo al programma e costituisce per ogni modulo la tabella dei simboli del modulo, questo avviene in 2 passaggi principali:

1. Traduce i codici mnemonici (`add`, `addi`, `sub`, `lw`, `sw`, `j`, `etc etc`) delle istruzioni nei corrispondenti codici binari
2. Traduce i riferimenti simbolici (variabili, registri, etichette di salto, parametri) nei corrispondenti indirizzi numerici

Ogni lettura del programma sorgente è chiamata passo e l'assemblatore è chiamato traduttore a due passi, ogni modulo assemblato di default parte dall'indirizzo 0, in sistemi dotati di meccanismi di memoria virtuale tali indirizzi sono indirizzi virtuali

Tabella dei Simboli

La tabella dei simboli contiene i riferimenti simbolici presenti nel modulo da tradurre e, al termine del primo passo, conterrà gli indirizzi numerici di tutti i simboli, tranne quelli esterni al modulo in esame.

Per le etichette associate a direttive dell'assemblatore che definiscono costanti simboliche nella tabella dei simboli viene creata la coppia **<etichetta, valore>** e in ogni istruzione che fa riferimento al simbolo viene sostituito il valore.

Per etichette che definiscono variabili (spazio di memoria + eventuale inizializzazione), l'assemblatore riserva spazio, eventualmente inizializza la zona di memoria e crea nella tabella la coppia **<etichetta, indirizzo>**. In ogni istruzione che fa riferimento al simbolo viene sostituito l'indirizzo.

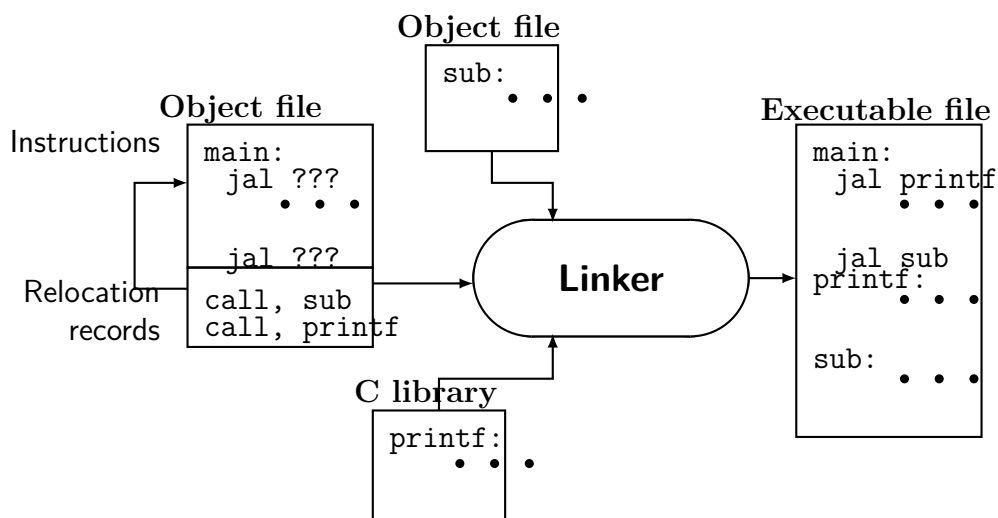
Nelle etichette presenti nelle istruzioni di salto, l'assemblatore deve generare un riferimento all'indirizzo dell'istruzione destinazione di salto

11.3.3 Linker (Link editor)

Cosa fa?

- Inserisce in memoria in modo simbolico il codice e i moduli dati
- Determina gli indirizzi dei dati e delle etichette che compaiono nelle istruzioni
- Corregge i riferimenti interni ed esterni e risolve i riferimenti in sospeso (a etichette esterne)
- Genera il file eseguibile
- Lettura dell'intestazione del file eseguibile per determinare la lunghezza del segmento di testo e del segmento dati
- Creazione di uno spazio di indirizzamento sufficiente a contenere testo e dati
- Copia delle istruzioni e dati dal file eseguibile in memoria
- Copia nello stack degli eventuali parametri passati al programma principale
- Inizializzazione dei registri e impostazione dello stack pointer affinché punti alla prima locazione libera
- Salto a una procedura di startup la quale copia i parametri nei registri argomento e chiama la procedura principale del programma
- Quando la procedura principale restituisce il controllo, la procedura di startup termina il programma con una chiamata alla funzione di sistema exit

Struttura



11.4 Convenzioni nomi ed uso dei registri

Nome simbolico	Numero	Uso
\$zero	0	Costante 0
\$at	1	"Variabile" temporanea dell'assembler
\$v0-\$v1	2-3	Valutazione di funzioni ed espressioni
\$a0-\$a3	4-7	Argomento di "funzione"
\$t0-\$t7	8-15	"Variabili" temporanee
\$s0-\$s7	16-23	Variabili temporanee salvate
\$t8-\$t9	8-15	"Variabili" temporanee
\$k0-\$k1	26-27	Riservati per il kernel del SO
\$gp	28	Global pointer
\$sp	29	Stack pointer
\$fp	30	Frame pointer
\$ra	31	Return address

- Usati da assembler, compilatore, sistema operativo
- Secondo specifiche **convenzioni**

11.5 Direttive all'assemblatore

11.5.1 Cosa sono?

Sono indicazioni date all'assembler per consentirgli di:

- Associare etichette simboliche a indirizzi
- Allocare spazio di memoria per le variabili
- Decidere in quali zone di memoria allocare istruzioni e dati
- ...

11.5.2 Esempi di direttive

- `.data <addr>`
 - Quel che segue va nel segmento dati (eventualmente dall'indirizzo `addr`)
- `.byte  $b_1, \dots, b_n$`
 - Inizializza i valori in byte successivi
- `.word  $w_1, \dots, w_n$`
 - Inizializza i valori in word successive
- `.text <addr>`
 - Quel che segue va nel segmento `text(programma)` (eventualmente dall'indirizzo `addr`)

11.5.3 Esempio d'uso

```
1 item:
2     .data
3     .word1
4     .text
5     .globl main
6
7 main:
8     la $t0, item      #carico l'indirizzo di memoria della variabile item in $t0
9     lw $t1, 0($t0)    #carico in $t1 il valore che si trova all'indirizzo
10                        #all'indirizzo di memoria 0 piu' il valore che sta nel registro
                        $t0
```

11.6 Pseudoistruzioni

11.6.1 Cosa sono?

Sono istruzioni assembly che non hanno una corrispondente istruzione macchina, per far si che il programma funzioni vengono tradotte dall'assembler in una sequenza di istruzioni

11.6.2 Esempio 1

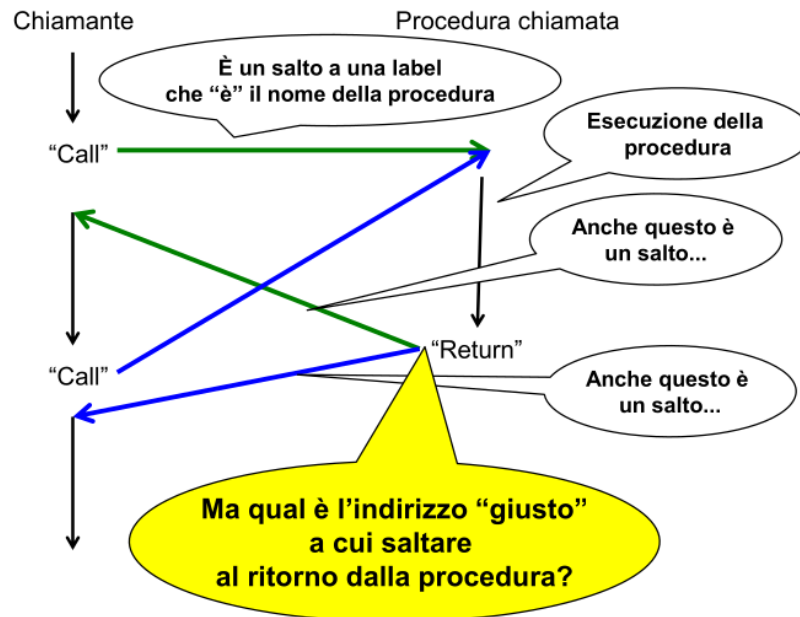
- `li rdest, imm`
 - load immediate
 - Carica una costante di 32 bit nel registro rdest
- ```
li $s0, 4000000
```

### 11.6.3 Esempio 2

- `move $t0, $t1`
  - Tradotta dall'assemblatore in `add $t0, $zero, $t1`

## 11.7 Flusso di controllo

### 11.7.1 Problema



### 11.7.2 Istruzione jal - Jump And Link

Cosa fa?

- Salta a una procedura indicata nell'istruzione e contemporaneamente crea un collegamento a dove deve ritornare per continuare l'esecuzione del chiamante
- Salva nel registro \$ra l'indirizzo a cui tornare dopo l'esecuzione della procedura

Esempio in assembly

```
1 .data
2 risultato: .word 0
3
4 .text
5 .globl main
6
7 main:
8 li $a0, 5 # Parametro di input: $a0 = 5
9 jal raddoppia # Chiamata a procedura: salta a 'raddoppia' e salva
 # PC+4 in $ra
10
11 rientro_main: # <-- $ra puntera' all'indirizzo di questa istruzione
12 !
13 sw $v0, risultato # Salva il risultato restituito ($v0) in memoria
14
15 # Chiusura del programma
16 li $v0, 10
17 syscall
18
19 # -----
20 # Procedura foglia 'raddoppia': calcola $v0 = $a0 * 2
21 # -----
22 raddoppia:
23 add $v0, $a0, $a0 # $v0 = $a0 + $a0 (raddoppia l'argomento)
24 jr $ra # Ritorno al chiamante: salta all'indirizzo
 # contenuto in $ra
```

## 11.7.3 Istruzione jr - Jump Register

Cosa fa?

- Salta all'indirizzo contenuto in un registro
- È una istruzione di uso generale che consente di saltare a qualsiasi locazione

Esempio in assembly

```
1 .data
2 stringa: .asciiz "Hello World!"
3 dim: .word 0
4 charfine:.asciiz ""
5
6 .text
7 .globl main
8
9 main:
10 la $a0, stringa
11 jal calcola_dim_stringa # Salta a calcola_dim_stringa e salva PC+4 in $ra
12
13 # Salva il risultato restituito in $v0
14 la $t7, dim
15 sw $v0, 0($t7)
16
17 # Termina l'esecuzione
18 li $v0, 10
19 syscall
20
21 # -----
22 # Procedura calcola_dim_stringa
23 # -----
24 .globl calcola_dim_stringa
25 calcola_dim_stringa:
26 add $t2, $zero, $zero
27 la $t5, charfine
28 lb $t1, 0($t5)
29
30 ciclo:
31 lb $t7, 0($a0)
32 beq $t7, $t1, fine
33 addi $t2, $t2, 1 # Incrementa la dimensione
34 addi $a0, $a0, 1 # Incrementa l'indirizzo per il prossimo char
35 j ciclo
36
37 fine:
38 move $v0, $t2 # Inserisce il valore di ritorno in $v0
39 jr $ra # <--- Ritorna al chiamante saltando all'indirizzo
 in $ra
```

## 11.8 Syscall

### 11.8.1 Cosa sono?

Sono delle operazioni analoghe a delle chiamate di procedura che permettono di emulare alcune funzionalità base del sistema operativo.

Le convenzioni per le le syscall si trovano a pagina A43 dell'appendice A

## 11.8.2 Come usare le syscall

1. Impostare nel registro `$v0` il codice della chiamata
2. Impostare i parametri nei registri `$a0-$a3`
3. `syscall`

## 11.8.3 Tabella

| Service                   | System call code | Arguments                                                                                        | Result                                    |
|---------------------------|------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>print_int</code>    | 1                | <code>\$a0 = integer</code>                                                                      |                                           |
| <code>print_float</code>  | 2                | <code>\$f12 = float</code>                                                                       |                                           |
| <code>print_double</code> | 3                | <code>\$f12 = double</code>                                                                      |                                           |
| <code>print_string</code> | 4                | <code>\$a0 = string</code>                                                                       |                                           |
| <code>read_int</code>     | 5                |                                                                                                  | integer (in <code>\$v0</code> )           |
| <code>read_float</code>   | 6                |                                                                                                  | float (in <code>\$f0</code> )             |
| <code>read_double</code>  | 7                |                                                                                                  | double (in <code>\$f0</code> )            |
| <code>read_string</code>  | 8                | <code>\$a0 = buffer</code> , <code>\$a1 = length</code>                                          |                                           |
| <code>sbrk</code>         | 9                | <code>\$a0 = amount</code>                                                                       | address (in <code>\$v0</code> )           |
| <code>exit</code>         | 10               |                                                                                                  |                                           |
| <code>print_char</code>   | 11               | <code>\$a0 = char</code>                                                                         |                                           |
| <code>read_char</code>    | 12               |                                                                                                  | char (in <code>\$a0</code> )              |
| <code>open</code>         | 13               | <code>\$a0 = filename (string)</code> , <code>\$a1 = flags</code> ,<br><code>\$a2 = mode</code>  | file descriptor (in <code>\$a0</code> )   |
| <code>read</code>         | 14               | <code>\$a0 = file descriptor</code> , <code>\$a1 = buffer</code> ,<br><code>\$a2 = length</code> | num chars read (in <code>\$a0</code> )    |
| <code>write</code>        | 15               | <code>\$a0 = file descriptor</code> , <code>\$a1 = buffer</code> ,<br><code>\$a2 = length</code> | num chars written (in <code>\$a0</code> ) |
| <code>close</code>        | 16               | <code>\$a0 = file descriptor</code>                                                              |                                           |
| <code>exit2</code>        | 17               | <code>\$a0 = result</code>                                                                       |                                           |

## 11.8.4 Esempio d'uso senza procedure

```
1 .data
2 stringa: .ascii "Hello World!"
3 #13 byte per memorizzare la stringa
4 #12 byte lunghezza della stringa
5 dim: .word 0
6 charfine: .ascii ""
7
8 .text
9 .globl main
10
11 main:
12 la $t0, stringa
13 add $t2, $zero, $zero
14 la $s5, charfine
15 lb $s1, 0($s5)
16
17 ciclo:
18 lb $s7, 0($t0)
19 beq $s7,options%keyvals, $s1, fine
20 addi $t2, $t2, 1
21 addi $t0, $t0, 1
22 j ciclo
23
24 fine:
25 la $t3, dim
26 sw $t2, 0($t3)
27
28 # scrivo la stringa sullo schermo
29 li $v0, 4
30 la $a0, stringa
31 syscall
32
33 # scrivo la dim della
34 # stringa sullo schermo
35 li $v0, 1
36 la $t0, dim
37 lw $a0, 0($t0)
38 syscall
```

## 11.8.5 Esempio d'uso con procedure

```
1 .data
2 stringa: .asciiz "Hello World!"
3 dim: .word 0
4 charfine: .asciiz ""
5
6 .text
7 .globl main
8
9 main:
10 la $a0, stringa
11 jal calcola_dim_stringa
12 # la dimensione della stringa
13 # e' in $v0
14 # salva la dim in memoria
15 la $t7, dim
16 sw $v0, 0($t7)
17 # scrive la stringa sullo schermo
18 li $v0, 4
19 la $a0, stringa
20 syscall
21
22 # scrive la dim della
23 # stringa sullo schermo
24 li $v0, 1
25 la $t0, dim
26 lw $a0, 0($t0)
27 syscall
28
29 .globl calcola_dim_stringa
30 calcola_dim_stringa:
31 add $t2, $zero, $zero
32 la $t5, charfine
33 lb $t1, 0($t5)
34
35 ciclo:
36 lb $t7, 0($a0)
37 beq $t7, $t1, fine
38 addi $t2, $t2, 1 # incrementa la dim
39 addi $a0, $a0, 1 # incrementa l'indirizzo
40 # per il prossimo elemento della stringa
41 j ciclo
42
43 fine:
44 move $v0, $t2
45 jr $ra
```

## Parte 2

### Esercitazioni svolte

# **Parte 3**

## **Laboratori svolti**

## Parte 4

### Quiz Mega svolti

## Parte 5

### Domande aperte Mega svolte